I M Smrti — How the Data Works, and Where It Breaks

For: Shreya **From:** Rohit **Updated:** 25 August 2026

---

1. What the product is

I M Smrti (imsmrti.app) — a health-record app for the person in an Indian family who manages everyone else's health. The son or daughter chasing prescriptions and lab reports for ageing parents across three different doctors.

You add a family member, photograph their reports, and the app:

- stores everything securely (12 Indian languages, DPDP-compliant)
- explains each report in plain spoken Hindi/Tamil/etc. — not textbook language
- generates a one-page briefing to hand the doctor before an appointment
- shows an emergency QR card (blood group, allergies, medicines) anyone can scan
- **turns reports into structured data, charts trends, and tells you where your

health is heading** ← the part this document is about

Stack: React + Firebase (Firestore) + Google Gemini. The app is live and working; the open problems are almost entirely data problems, not app problems.

---

2. How data actually flows

```
photo/PDF upload
      ↓
Gemini (one call) ──► returns BOTH:
      │                  • a plain-language summary for the human
      │                  • structured JSON: metrics / medicines / follow-ups
      ↓
validation guard  ──► rejects anything that isn't a real medical measurement
      ↓
Firestore  ──► health_metrics · medications · follow_ups
      ↓
      ├─► trend charts (per test, per patient)
      ├─► health trajectory engine (score, direction, suggested actions)
      └─► daily background agent (abnormal results, overdue follow-ups, refills)
```

The collections

`health_metrics` — one row per lab value

```
{
  "userId":"...", "patientId":"...",        // account + WHICH family member
  "documentId":"...", "documentName":"Ritu test 2026.pdf",
  "test":"Vitamin D",                        // canonical name — groups the trend
  "testRaw":"VITAMIN D (25-OH)",             // exactly as printed
  "value":8.4, "unit":"ng/ml",
  "refLow":30, "refHigh":100,
  "status":"low",                            // low | normal | high | unknown
  "date":"2026-02-05"
}
```

`medications` — `{ name, dose, frequency, durationDays, startDate, expectedEndDate }` `follow_ups` — `{ advice, dueDate, status }` (drives "your review is 12 days overdue") `patients` — `{ name, gender, dob, bloodGroup, allergies, conditions, relationship }`

Every record carries both `userId` and `patientId`. Mixing patients would be a serious bug — a father's cholesterol must never appear on his daughter's chart.

---

3. The validation guard (and why it exists)

A business PDF — a client pricing plan — was once mined for numbers, and **₹60,000 / ₹1,20,000 / ₹3,00,000 were saved as "Weight" readings in INR** and charted. Separately, a Vitamin D result was stored with a reference range of `null–1` when the real range is 30–100 ng/mL.

That's the kind of failure that makes someone stop trusting a health app entirely, so there's now a guard that every extracted number must pass:

- **Non-clinical documents produce zero metrics.** No exceptions.
- **Currency, phone numbers, IDs, durations, counts** are rejected outright.
- **Physiological plausibility** — a 450 kg body weight or a 9-digit "value" is

thrown out.

- **Bogus reference ranges get repaired** from a trusted table rather than trusted

blindly from the model.

It's verified against 12 cases including the exact garbage that reached production. It passes all 12.

Important limitation: a guard only *blocks bad data*. It does nothing to *produce good data*. That's problem #1 below.

---

4. The genuinely hard problems (the interesting bit)

Problem 1 — Coverage, not accuracy

Of 48 documents: **26 were never AI-analysed at all**, 12 are structured, and 10 have summaries that haven't been converted to structured data. I originally assumed extraction was broken; the real gap is that most documents never went through the pipeline. Backfill now runs in batches of 40 and continues until done — but the 26 unanalysed ones need fresh AI calls against the original images, which costs money per document.

Open question: what's the right re-processing strategy when a user arrives with 200 old reports? Batch overnight? Charge for it? Process only what's recent?

Problem 2 — No canonical medical vocabulary

The same test is printed differently by every lab, so trend lines silently split into separate series instead of forming one:

```
"HBA1C" · "HbA1c" · "Glycated Haemoglobin" · "A1c"        → must be ONE concept
"VITAMIN D (25-OH)" · "Vit D3" · "25-Hydroxyvitamin D"    → ONE concept
```

I've hand-built **66 tests** (attached: `lab_vocabulary_current.csv`) covering CBC, lipids, LFT, KFT, thyroid, sugar, vitamins, electrolytes, urine routine. Each has: canonical name, accepted units, normal range, plausibility bounds, and a plain-language explanation.

Two things I know are weak:

1. **Ranges are adult-generic.** They don't vary by age or sex, and they genuinely

should (Hemoglobin, Ferritin, Creatinine, HDL all differ by sex; children differ a lot). In a health app a wrong "normal" is dangerous — it can make a serious value look fine.

1. **They're my best knowledge, not clinically reviewed.** This needs a doctor or

pathologist to sign off, not an engineer.

Problem 3 — Drug names are a mess

Real strings sitting in the database right now:

Extracted as	Probably is	Issue
`Vitad3 60K`	Cholecalciferol 60,000 IU	dose buried in brand name
`Mouture LC`	Montair/Montek LC (Montelukast + Levocetirizine)	OCR error
`Grahaconeazole`	likely Itraconazole	OCR error, unusable
`Amrol Fin`, `Zine 200 mg`, `Contimega O Lo`	?	unresolved brands

`dose` and `frequency` come back empty most of the time. Mapping Indian brand names → generic molecule + strength is a real data problem I haven't solved.

Problem 4 — Accuracy is unmeasured

There is **no ground truth set**, so nobody can say whether extraction is 40% accurate or 90%. That's precisely how the rupee-amount bug survived. The fix is unglamorous: 30–50 real reports, hand-labelled, used as a ruler. Without it, every "improvement" is a guess.

---

5. What's already built on top

- **Trend charts** per test per patient, with reference bands and source document
- **Health trajectory engine** — computes a 0–100 attention score, whether each

value is moving toward or away from the healthy range, and suggests concrete actions grouped by what shares a response (sugar / cholesterol / vitamins / anaemia / kidney / liver / thyroid / BP). Purely computed from the person's own records — no AI guessing on the numbers.

- **Daily background agent** — flags abnormal results, worsening 3-reading trends,

overdue doctor follow-ups, and medicines about to run out.

---

6. Where the data layer goes from here

As volume grows, three pieces of real data engineering sit ahead:

1. **Medical vocabulary at scale** — LOINC mapping, drug normalisation, age/sex-aware

ranges. The highest-value piece, and the one everything else depends on.

1. **Accuracy measurement + monitoring** — gold-standard sets, precision/recall per

test, dashboards showing which labs and document formats fail most.

1. **De-identification + aggregation** — the ambition I'm most excited about:

understanding patterns in Indian health at population scale (which conditions by age group, which medicines, which deficiencies) and eventually putting that in front of researchers or public-health bodies. Done properly it means k-anonymity, minimum group sizes, banded ages, no free text — real privacy engineering.

On that last one, the legal reality: our privacy policy says data is *"strictly used to provide medical record storage… generate AI health insights."* Research is a **different purpose** under DPDP 2023 and needs **separate opt-in consent** — which we're adding, off by default. And "without a name" is not anonymous: age + city + a rare condition re-identifies people. So this gets built correctly at proper scale, not as a shortcut.

---

7. The piece I'd most want designed properly

Everything above depends on one thing: **the canonical vocabulary system.** Right now it's a flat hand-written table (attached). It works, but it's the weakest link in the whole pipeline, and it's a data-modelling problem rather than an app problem.

What exists today

One row per test — canonical name, pipe-separated aliases, one unit, one adult reference range, a plain-language explanation. Lookup is exact match, then a prefix fallback.

What it can't currently do

Gap	Why it matters
Aliases are hand-listed	A lab prints `S. CREAT.` or `Creat (Serum)` and we miss the match entirely — the trend line silently splits in two
One range for everyone	Hemoglobin, Ferritin, Creatinine, HDL and Uric Acid genuinely differ by sex; children differ a lot. A wrong "normal" can make a serious result look fine
No unit conversion	`mg/dL` vs `mmol/L` vs `g/L` for the same test are treated as unrelated
Drug names unsolved	`Vitad3 60K` , `Mouture LC` , `Grahaconeazole` — Indian brands, OCR errors, dose buried in the name
No confidence signal	Every match is treated as certain, so a bad match looks identical to a good one

The design questions worth thinking through

1. **Alias resolution** — exact table, normalised string matching, fuzzy/edit-distance,

or embeddings? Where's the right accuracy/complexity trade-off for messy OCR text?

1. **Schema for variation** — how should age and sex ranges be modelled so lookup stays

simple? Rows per demographic, or ranges as nested rules?

1. **Units** — normalise on write or convert on read? Where do conversion factors live?
2. **Drug normalisation** — Indian brand → molecule + strength. Lookup table, fuzzy

matcher, an LLM step, or a combination? What happens when confidence is low?

1. **Measuring it** — how would you prove any version of this is better than the last?

If you want something concrete to build

Restructure the attached CSV into whatever schema you think it *should* be, and describe how lookup would work against it. That artefact alone — the schema plus the matching strategy — would be more valuable than anything else on this list, and it transfers directly into the app.

Honest views are worth more than polite ones. If you think the whole approach is wrong, that's the most useful thing you could tell me.

Attached: `lab_vocabulary_current.csv` — the 66 tests, ranges and plain-language explanations currently powering the app.